

Informe técnico de Aikukisna

1 Propósito y alcance

Este informe explica el diseño, la implementación Android, el backend Supabase, el modelo relacional, la normalización y el funcionamiento sin conexión de Aikukisna. Está orientado al equipo que necesita estudiar y defender las decisiones del proyecto ante un jurado o una revisión técnica.

Fecha de corte: 1 de octubre de 2026, zona America/Guatemala. Se revisó el código de trabajo existente, incluidos cambios aún no confirmados en Git, y se consultaron en modo lectura los metadatos del proyecto Supabase activo Aikukisna. No se realizaron cambios en la base remota ni una nueva compilación para producir este informe.

La explicación separa tres niveles: **implementado**, cuando hay código o estructura verificable; **validado**, cuando existe evidencia de una prueba; y **pendiente**, cuando la capacidad todavía no está terminada o requiere comprobación. Tener una tabla, una dependencia o una pantalla no demuestra por sí solo que todo su flujo esté operativo.

Conclusión central: Aikukisna es una aplicación Android nativa desarrollada principalmente en Kotlin, con interfaz Jetpack Compose, separación por capas, contenido local en Room sobre SQLite y backend administrado Supabase sobre PostgreSQL. Tiene funciones educativas, diccionario, traducción documentada, cámara, voz, tutoría y gamificación. La operación offline es real para determinados flujos, pero no toda la aplicación aplica una política estricta de lectura local primero. Tampoco incorpora todavía un traductor neuronal libre Miskito/Kriol en el APK.

El README histórico menciona 16 tablas y Gemini 2.5 Flash. Esa descripción no representa íntegramente el estado inspeccionado. Este informe utiliza el esquema vivo y el código actual, no esas cifras antiguas.

2 Qué problema atiende la aplicación

Aikukisna aborda el aprendizaje de lenguas y el acceso a contenido cultural en un contexto donde la conexión puede ser intermitente. La motivación documentada incluye estudiantes de comunidades miskitas y su transición a entornos escolares hispanohablantes, sin perder la lengua de origen. El modelo también incorpora Kriol e inglés estándar.

Los cuatro registros actuales de idioma son Miskito, español, inglés Kriol e inglés estándar. Los códigos internos son *mi*, *es*, *jam* y *en*, respectivamente. Estos identificadores son convenciones observadas del proyecto: no deben presentarse automáticamente como una clasificación lingüística universal ni como prueba de soporte de un proveedor externo.

El producto reúne aprendizaje estructurado, consulta lingüística, práctica y seguimiento. Las lecciones organizan contenidos; el diccionario recupera entradas; el Traductor busca equivalencias; Cámara vincula objetos o texto con conocimiento; Tuki orienta al estudiante; perfil, favoritos, XP, rachas y logros conservan su actividad.

Objetivo no equivale a capacidad entregada. La meta de traducción conversacional bidireccional es más amplia que el motor actual, que recupera palabras, expresiones y oraciones registradas. Una interfaz con dos idiomas y botón de intercambio no implica traducción libre de cualquier frase.

3 Inventario de tecnologías

Las versiones siguientes se leen del catálogo Gradle y de la configuración del proyecto; no son una recomendación de versiones más recientes.

Tecnología	Versión o configuración observada	Función concreta
Kotlin	2.2.10 en el catálogo	Pantallas, ViewModels, dominio, repositorios y utilidades
Android Gradle Plugin	9.2.1	Construcción de la aplicación Android
Gradle Wrapper	9.4.1	Ejecución reproducible de tareas de construcción
Android SDK	minSdk 26, compileSdk 37, targetSdk 37	Compatibilidad mínima y nivel objetivo configurado
Compatibilidad Java	VERSION_11	Compatibilidad del código JVM; no es la versión del JDK que ejecuta Gradle
Jetpack Compose	BOM 2026.02.01	Interfaz declarativa nativa
Material 3	Gestionado por Compose BOM	Componentes y tema visual
Navigation Compose	2.9.8	Rutas y navegación entre pantallas
Lifecycle	2.11.0	Integración de estado y ciclo de vida
Hilt	2.60.1	Inyección de dependencias
KSP	2.2.10-2.0.2	Generación de código de Room e Hilt
Room	2.8.4, esquema local versión 12	Persistencia estructurada sobre SQLite
Supabase Kotlin	BOM 3.6.0	Auth, PostgREST y Functions
Ktor con motor OkHttp	3.5.0	Transporte HTTP del cliente
Kotlin Serialization	Dependencia declarada	Conversión JSON y DTO
CameraX	1.5.3	Captura y vista previa de cámara
ML Kit OCR	16.0.1	Reconocimiento local de texto latino
ML Kit Image Labeling	17.0.9	Etiquetas visuales locales
ML Kit Object Detection	17.0.2	Regiones de objetos
Vosk Android	0.3.75	Reconocimiento de voz local con modelos español e inglés
JNA	5.18.1	Interoperabilidad nativa requerida por dependencias de voz
WorkManager	2.11.2	Trabajo persistente condicionado por conectividad
Credential Manager	1.5.0	Integración de credenciales Android
Google ID	1.1.1	Inicio de sesión Google
JUnit	4.13.2	Pruebas unitarias
AndroidX JUnit y Espresso	1.3.0 y 3.7.0	Infraestructura declarada de pruebas Android
PostgreSQL	17.6.1.141 reportado por Supabase	Base relacional central

Tecnología	Versión o configuración observada	Función concreta
TypeScript y Deno	Edge Functions desplegadas	Autenticación auxiliar y proxies de IA/voz
Gemini	Endpoint configurado gemini-3.5-flash-lite	IA remota mediante gemini-proxy
ElevenLabs	Modelo configurado eleven_multilingual_v2	Síntesis remota mediante sintetizar-voz

El identificador de Gemini se verificó en el código desplegado del proxy, no mediante una nueva inferencia. Su presencia no demuestra disponibilidad, cuotas o calidad actual del proveedor.

El directorio `app/src/main` contiene 247 archivos Kotlin y ningún archivo `.java` en el corte observado. Aunque Android utiliza herramientas del ecosistema Java/JVM, no es correcto describir el código propio principal como una aplicación escrita en Java. Kotlin se compila a código compatible con la cadena Android y finalmente a DEX, ejecutado por Android Runtime; no se ejecuta en una JVM de escritorio dentro del teléfono.

No se encontró una implementación Kotlin Multiplatform ni un frontend Flutter, React Native o web. `settings.gradle.kts` incluye un único módulo Gradle `:app`: la separación es principalmente por paquetes, no por múltiples módulos compilables ni microservicios propios.

3.1 Tecnologías de preparación y entrenamiento

Fuera del APK existen scripts Python para extracción, revisión de corpus y entrenamiento. `requirements-traducción.txt` fija PyTorch 2.9.1, Transformers 4.57.6, Datasets 4.8.5, Accelerate 1.12.0, SacreBLEU 2.6.0 y psutil 7.2.2. El experimento usa `google/byt5-small` y contempla entrenamiento bidireccional, particiones independientes y evaluación por dirección.

Estas herramientas no son motores integrados en Android. No debe afirmarse que la app ya ejecuta ByT5, llama.cpp, Whisper, YOLO, MediaPipe, ONNX Runtime o un LLM local porque se hayan considerado en una propuesta. La implementación visible usa ML Kit, Vosk y motores de respuesta locales basados en conocimiento y reglas, además de servicios remotos.

4 Arquitectura y organización del desarrollo

4.1 Capas y responsabilidades



presentacion/pantallas contiene los composables; presentacion/viewmodel coordina acciones y estado; domain/model, domain/repository y domain/usecase expresan conceptos, contratos y operaciones; data/local implementa persistencia y capacidades del dispositivo; data/remote/dto representa mensajes remotos; data/repository combina las fuentes; data/sync gestiona sincronización; di conecta implementaciones con interfaces.

Es una arquitectura inspirada en Clean Architecture y MVVM. MVVM separa la vista del estado y las acciones del ViewModel. Clean Architecture intenta que las reglas no dependan de detalles de infraestructura. En Aikukisna esa separación existe, pero no es absoluta: por ejemplo, TranscribirAudioUseCase importa clases concretas de data.local, y algunos ViewModels conocen el sembrador local. La defensa precisa es «separación por capas con contratos y casos de uso, con acoplamientos concretos todavía presentes», no «independencia perfecta de frameworks».

Hilt evita construir manualmente la cadena de dependencias en cada pantalla. NetworkModule proporciona un cliente Supabase singleton; DatabaseModule proporciona Room; RepositoryModule vincula interfaces con clases concretas mediante @Binds. El ViewModel puede pedir un caso de uso sin crear por su cuenta un cliente HTTP o una base de datos.

4.2 Por qué hay tres representaciones del dato

Un DTO representa el JSON remoto, por ejemplo idioma_id y relaciones embebidas de PostgREST. Una Entity representa las columnas locales de Room. Un modelo de dominio expresa el objeto que consume el negocio o la interfaz. Los mapeadores traducen entre estas formas.

No es duplicación sin propósito: cada forma tiene un contrato distinto. Si PostgreSQL cambia un campo o Room necesita un índice, se puede adaptar la capa de datos sin rediseñar todas las pantallas. El costo es mantener mapeadores y comprobar que los campos no se pierden.

4.3 Concurrencia y presentación

Se usan funciones suspend, coroutines, viewModelScope, Dispatchers.IO y estado observable. La aplicación inicializa contenido con SupervisorJob y comunica Preparando, Disponible o NoDisponible. No toda pantalla usa exclusivamente StateFlow: TraductorViewModel, por ejemplo, utiliza mutableStateOf.

El Traductor captura texto e idiomas al iniciar una consulta y descarta el resultado si ya no corresponde a la selección actual. Esto evita mostrar una respuesta vieja después de cambiar de idioma. La cancelación se vuelve a lanzar en ese flujo para no tratarla como un error lingüístico.

5 Diseño de interfaz y experiencia

La UI es nativa declarativa. Las pantallas describen qué mostrar según el estado, en vez de modificar manualmente cada vista. GrafoNavegacion.kt organiza las rutas y el punto de entrada según el estado de la aplicación.

Existen pantallas de inicio, onboarding, autenticación, recuperación de contraseña, selección de idioma, perfil, lecciones, vocabulario, cuestionario, resultados, diccionario, detalle de palabra, favoritos, logros, cultura, Tuki, Traductor, Cámara y descarga offline. También hay un recorrido de invitado. La existencia de estos archivos documenta componentes implementados, no una aprobación funcional total de cada recorrido.

El tema define modo claro y oscuro, naranja principal #FD8B23, azul #1579D1, colores semánticos y tipografías Nunito, Agu Display y JetBrains Mono. Los comentarios del tema mencionan referencias de Figma; el archivo de diseño original no se inspeccionó en este informe. Por tanto, se puede explicar la implementación de tokens visuales, no certificar fidelidad completa al prototipo.

Tuki, mapas de lecciones, ilustraciones, XP y rachas articulan la experiencia educativa. El propósito defendible es facilitar orientación y continuidad del aprendizaje. No hay evidencia aquí para atribuir mejoras porcentuales de aprendizaje, accesibilidad certificada o eficacia pedagógica validada experimentalmente.

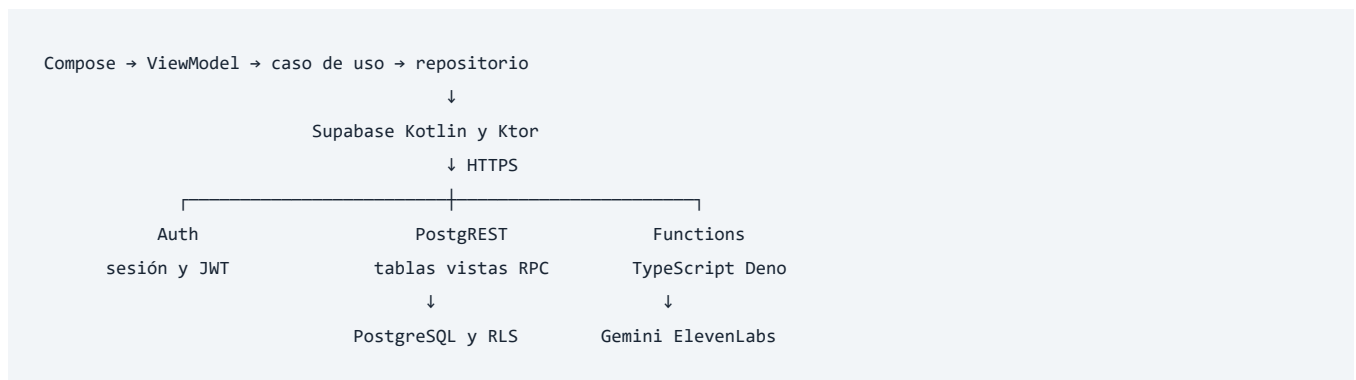
La app solicita permisos de cámara y micrófono para esas funciones, además de acceso a red y estado de conectividad. La cámara está declarada como hardware no obligatorio, de modo que no define por sí sola la compatibilidad de instalación. La calidad de OCR, voz y detección depende del dispositivo, iluminación, audio y disponibilidad de modelos.

6 Backend Supabase y conexión con Android

6.1 Qué es frontend y qué es backend en este proyecto

El frontend es la aplicación instalada: Compose, ViewModels y la interacción de la persona. La capa data de Android es infraestructura del cliente, no el servidor remoto. El backend está compuesto por Supabase Auth, PostgreSQL, PostgREST, funciones SQL y Edge Functions.

No se requiere que Android abra una conexión PostgreSQL directa ni que reciba credenciales de administración de la base. El SDK Supabase realiza peticiones HTTPS. PostgREST expone operaciones autorizadas sobre tablas, vistas y funciones; Auth emite la sesión; PostgreSQL aplica políticas y restricciones. Este mecanismo de API generada está documentado por [Supabase Data REST API](#).



`NetworkModule` instala Postgrest, Auth y Functions, configura un tiempo de espera de 30 segundos y el retorno de autenticación `aikukisna://auth-callback`. `SUPABASE_URL` y `SUPABASE_ANON_KEY` se obtienen de `local.properties` durante la construcción y se exponen en `BuildConfig`.

La clave pública identifica el proyecto; no convierte al cliente en administrador. El JWT de la sesión identifica a la persona. No son intercambiables y no sustituyen las políticas RLS. Ningún ejemplo de este informe contiene claves, contraseñas o tokens reales.

6.2 Ejemplo de consulta y respuesta

En `LeccionRepositoryImpl`, una lectura usa `client.from("leccion").select(...)`, solicita datos de categoría e idioma relacionados, aplica filtros y decodifica `LeccionDto`. Después convierte a dominio y guarda una copia local. Las relaciones embebidas de PostgREST aprovechan las claves foráneas de PostgreSQL.

Representación conceptual de la petición:

```
GET /rest/v1/leccion?select=*,categoria(*),idioma_meta:idioma_meta_id(*)&nivel=eq.1
apikey: <clave_publica_del_proyecto>
Authorization: Bearer <jwt_de_la_sesion_cuando_corresponda>
```

La respuesta JSON vuelve al repositorio, se transforma en modelos y se publica como estado. La pantalla no debe decidir las reglas de integridad remota ni contener SQL de PostgreSQL.

6.3 Autenticación

El repositorio implementa correo y contraseña mediante Supabase Auth, Google mediante ID token, recuperación y cambio de contraseña, cierre de sesión e importación de sesión. Para nombre de usuario utiliza login-usuario: el servidor resuelve internamente el correo y valida la contraseña con Auth; el cliente recibe tokens tras un inicio válido, no una API pública de búsqueda de correos.

La Edge Function de login está desplegada con `verify_jwt=false`, coherente con su uso antes de iniciar sesión. Esto no significa que deba confiar en cualquier petición: su código comprueba el formato, la clave pública del cliente, longitudes y credenciales, y usa una respuesta genérica ante fallos. La clave pública no es un secreto ni una defensa contra abuso por sí sola; la resistencia a intentos masivos requiere controles adicionales y no fue certificada aquí.

`public.usuario.id` referencia `auth.users.id`. Auth gestiona identidad y credenciales; `usuario` contiene el perfil educativo. La función `crear_usuario_nuevo` inserta el perfil a partir de la identidad creada. La relación evita usar el correo como clave estable y permite relacionar progreso y favoritos con el mismo UUID.

6.4 Funciones de backend presentes

Se observaron tres Edge Functions activas: `gemin-proxy` versión 15, `synthetizar-voz` versión 5 y `login-usuario` versión 4. Los dos proxies exigen autenticación y comprueban un usuario real con Auth. Las claves de proveedores se leen de variables del servidor. El proxy de IA valida tamaños, roles del historial y tipos MIME; el de voz limita el texto a 1000 caracteres.

Hay 19 funciones SQL propias en `public`, sin contar las de extensiones. Entre ellas están `completar_leccion`, `desbloquear_logro`, `resolver_consulta_linguistica`, `traducir_texto_seguro`, `responder_tuki`, `obtener_contexto_tuki_usuario` y `obtener_paquete_leccion`. Que una función esté disponible en Supabase no implica que una pantalla Android la invoque: el Traductor actual resuelve localmente; Tuki remoto sí tiene un proxy que consulta RPC documentales.

No se observó instalación del módulo Realtime en `NetworkModule`: la sincronización descrita es mediante solicitudes y trabajo programado, no suscripciones en tiempo real. Tampoco se confirmó aquí una arquitectura propia de almacenamiento con Supabase Storage. No deben atribuirse todos los productos de Supabase a la aplicación solo por usar la plataforma.

7 Diseño real de la base de datos remota

7.1 Tamaño y naturaleza del esquema

La consulta de catálogo encontró **44 tablas públicas, 78 restricciones FK, 22 vistas, 135 índices y 47 políticas RLS**. Las 44 tablas tienen RLS habilitada. Los esquemas internos administrados por Supabase, como auth, no están incluidos en el número de tablas públicas.

Conteos exactos de registros al consultar: 4 idiomas, 54 fuentes documentales, 72 111 palabras, 132 350 traducciones, 72 925 acepciones, 8 896 expresiones, 1 010 oraciones y 93 lecciones. No son conteos de datos certificados: incluyen los estados almacenados y no miden por sí mismos cobertura, calidad o ausencia de duplicados semánticos. Tampoco equivalen al número de ejemplos aptos para entrenar un traductor.

7.2 Por qué existen tantas tablas

Las tablas representan hechos con identidad, cardinalidad y ciclo de vida distintos. Una palabra no es una traducción; una traducción no es una acepción; una lección no es el avance de una persona; una fuente bibliográfica no es la evidencia específica que respalda una entrada.

Guardar todo en una sola tabla multiplicaría datos: el título de una fuente se repetiría por cada palabra, el nombre de idioma por cada traducción y la descripción de una lección por cada estudiante. Además, limitaría las relaciones a un número fijo de columnas, como `traduccion_1`, `traduccion_2` o `idioma_3`.

La descomposición permite agregar idiomas, reutilizar contenido, relacionar varias fuentes y conservar distintos sentidos sin rediseñar cada pantalla. El costo es más joins, restricciones, migraciones y sincronización. Por ello, **más tablas no significa automáticamente mejor normalización**: cada una debe tener una responsabilidad y una relación justificables.

7.3 Catálogo funcional de las 44 tablas

Tabla	Qué representa y por qué está separada
idioma	Catálogo común de lenguas; evita repetir código y nombre en cada recurso
categoria	Clasificación reutilizable de palabras, lecciones y logros
fuente_documento	Identidad bibliográfica, URL, licencia y notas de uso
palabra	Entrada léxica de una lengua, pronunciación y estado documental
acepcion	Un sentido de una palabra; permite polisemia sin duplicar toda la entrada
variante_palabra	Forma ortográfica, dialectal, histórica u otra variante vinculada
palabra_canonica	Vinculación de una entrada con una forma canónica y su motivo
palabra_fuente	Relación palabra–documento con páginas y nota de evidencia
traduccion	Equivalencia entre dos palabras con estado, confianza y preferencia
traduccion_acepcion	Equivalencia entre sentidos concretos, no solo grafías
traduccion_fuente	Documento y tipo de respaldo de una traducción
expresion	Unidad de varias palabras con significado propio y lengua definida

Tabla	Qué representa y por qué está separada
traduccion_expresion	Equivalencia entre expresiones completas
oracion_ejemplo	Par textual contextualizado con idiomas, fuente y lección opcional
regla_gramatical	Explicación estructurada de una regla por idioma
ejemplo_regla_gramatical	Ejemplo de aplicación de una regla, con evidencia
regla_pronunciacion	Patrón y descripción fonética u ortográfica documentada
cultura_contenido	Texto cultural con procedencia y rango de páginas
expresion_contexto_cultural	Asocia una expresión con contexto cultural; su PK actual es solo expresion_id
revision_linguistica	Seguimiento de revisión de recursos y persona revisora
leccion	Identidad educativa base, nivel, capítulo, categoría e idioma meta
leccion_palabra	Reutilización de palabras en distintas lecciones
leccion_expresion	Expresiones usadas en una lección y tipo de vínculo
leccion_oracion	Reutilización de oraciones entre lecciones
leccion_fuente	Bibliografía asociada a una lección
leccion_cultura	Contenido cultural asociado a una lección
leccion_regla_gramatical	Reglas que se enseñan en la lección y orden
leccion_regla_pronunciacion	Reglas de pronunciación que se practican y orden
leccion_experiencia_gamificada	Presentación de la lección, mundo, mensajes y capacidades visibles
actividad_leccion	Pasos de una lección: tipo, orden, habilidad, mecánica y XP
actividad_recurso	Recursos que usa una actividad y papel de cada uno
mundo_gamificado	Organización visual y temática de conjuntos de lecciones
etapa_ruta_curricular	Etapas curriculares, modalidad y grados
leccion_ruta_curricular	Ubicación de una lección en una etapa y justificación
alineacion_curricular	Correspondencia temática y documental con referencias curriculares
evidencia_curricular_leccion	Evidencia detallada de competencias, indicadores y adecuación
usuario	Perfil educativo vinculado a Auth, idioma objetivo y agregados de actividad
progreso_leccion	Estado y resultado de una lección para una persona
palabra_favorita	Relación personal entre usuario y palabra
logro	Definición de una recompensa y su condición
logro_desbloqueado	Obtención de un logro por una persona y fecha
memoria_tuki	Resúmenes de interacción asociados a usuario
tuki_respuesta_sistema	Respuestas configurables, activación, prioridad e idioma
super_administrador	Membresía explícita del rol de administración de contenido

No todas las tablas son de aprendizaje visible. Algunas existen para procedencia documental, administración, trazabilidad o composición curricular. Tampoco todas se replican directamente en Android: `revision_linguistica` y `super_administrador`, por ejemplo, no forman parte del mapa de contenido público importado por el sembrador principal.

7.4 Relaciones y cardinalidades que conviene defender

```
idioma 1 — N palabra 1 — N acepcion
      |           |
      | traduccion — dos niveles de equivalencia separados
```

```
palabra N — M fuente_documento    mediante palabra_fuente
leccion N — M palabra              mediante leccion_palabra
leccion N — M expresion            mediante leccion_expresion
leccion 1 — N actividad_leccion
actividad_leccion 1 — N actividad_recurso
mundo_gamificado 1 — N leccion_experiencia_gamificada
```

```
auth.users 1 — 0..1 usuario
usuario N — M leccion              mediante progreso_leccion
usuario N — M palabra              mediante palabra_favorita
usuario N — M logro                mediante logro_desbloqueado
usuario 1 — N memoria_tuki
```

La primera sección del dibujo resume dos niveles: `traduccion` tiene dos FK a `palabra`; `traduccion_acepcion` tiene dos FK a `acepcion`. No hay una FK de `traduccion` a `acepcion`.

En `traduccion`, origen y destino son roles diferentes de la misma entidad. Una palabra puede participar en muchas relaciones en ambos roles. El índice único del par impide repetir exactamente la misma pareja orientada; no impide almacenar también la relación inversa ni asegura que ambas tengan la misma interpretación lingüística.

`leccion_experiencia_gamificada` y `leccion_ruta_curricular` tienen `leccion_id` como PK: son extensiones de una lección con como máximo una fila por lección. `expresion_contexto_cultural` tiene PK `expresion_id`, por lo que actualmente admite como máximo un contexto registrado por expresión en esa tabla; no es una tabla N:M general pese a su nombre.

Las FK con `ON DELETE CASCADE` eliminan dependencias sin identidad independiente, como un vínculo de lección. `SET NULL` permite conservar información cuyo padre opcional se elimina. Las FK sin acción de borrado especial impiden dejar referencias inválidas bajo las reglas ordinarias de PostgreSQL. Estas decisiones deben revisarse antes de borrar fuentes o contenido central; no constituyen una invitación a hacer borrados en producción.

7.5 Vistas e índices

Las 22 vistas ofrecen proyecciones de lectura, como `vista_diccionario_respuestas`, `vista_tuki_conocimiento`, `vista_actividad_recurso_resuelto` y `vista_catalogo_offline_lecciones`. Una vista ordinaria guarda una consulta, no necesariamente una copia física de sus resultados. Todas las vistas inspeccionadas tienen `security_invoker=true`, por lo que se evalúan con los permisos del invocador en lugar de usar indiscriminadamente los del propietario.

Hay índices B-tree de identidad, unicidad, relaciones y búsqueda, además de un GIN trigram sobre `palabra.texto.usuario_nombre_usuario_unico` es un índice único sobre `lower(nombre_usuario)`: impide nombres iguales salvo capitalización. Los índices aceleran determinados accesos y tienen un costo de escritura y espacio; no hacen rápidas todas las consultas automáticamente.

La búsqueda local revisada usa texto normalizado, igualdad, LIKE, prefijos y distancia de Levenshtein para sugerencias. No se encontró una tabla Room FTS5 implementada. No debe confundirse el GIN trigram remoto con una búsqueda FTS5 local.

8 Cómo explicar la primera segunda y tercera forma normal

8.1 Qué se puede demostrar

La normalización se explica mediante dependencias funcionales, claves y semántica, no por el número de tablas. El esquema observado permite reconstruir una descomposición razonada del núcleo. No se encontró una memoria histórica completa que pruebe cada paso original de diseño; el ejemplo siguiente es una reconstrucción didáctica fiel a las estructuras actuales, no una cronología inventada.

Una dependencia $x \rightarrow y$ significa que cada valor de X determina un único valor de Y según las reglas del dominio. Una clave candidata identifica una fila sin atributos sobrantes. La clave primaria es la candidata elegida; pueden existir otras, expresadas con UNIQUE o índices únicos.

8.2 Primera forma normal

Imaginemos un registro inicial con una palabra, una lista de traducciones, varias fuentes y una lista de lecciones. Las listas dentro de una celda dificultan relacionar y restringir cada elemento. Para llegar a 1FN se representan valores del dominio en columnas y cada relación repetible en filas propias.

En Aikukisna una traducción es una fila de `traduccion`; una fuente adicional es una fila de `palabra_fuente`; una asociación educativa es una fila de `leccion_palabra`. No hacen falta columnas `fuentes1`, `fuentes2` o `traduccion3`.

«Atómico» no significa una sola palabra escrita: el texto de una oración puede ser un valor textual atómico para el propósito de la tabla. Tampoco un JSON es automáticamente incorrecto; depende del modelo y las operaciones, aunque dificulta restricciones relacionales sobre sus componentes.

8.3 Segunda forma normal

En una relación hipotética `DetalleLeccion(leccion_id, palabra_id, titulo_leccion, texto_palabra, idioma_id)`, la clave es el par de identificadores. Sin embargo, `titulo_leccion` depende solo de `leccion_id`, y `texto_palabra` e `idioma_id` dependen solo de `palabra_id`. Son dependencias parciales de una clave compuesta.

La descomposición correcta es:

```
leccion(leccion_id, titulo, ...)
palabra(palabra_id, texto, idioma_id, ...)
leccion_palabra(leccion_id, palabra_id)
```

Ahora la tabla puente solo guarda la asociación. Este mismo argumento justifica `palabra_favorita(usuario_id, palabra_id)` y `logro_desbloqueado(usuario_id, logro_id, fecha)`. La fecha de desbloqueo pertenece al par, no al catálogo de logros ni a todos los usuarios.

8.4 Tercera forma normal

Si `palabra` incluyera `idioma_id`, `nombre_idioma` y `codigo_idioma`, existiría `palabra.id → idioma_id → nombre_idioma, codigo_idioma`. Los datos de idioma dependerían transitivamente de la palabra. Se separan en idioma y se conserva la FK.

Lo mismo ocurre con `fuelle_id → titulo, autor, anio, institucion` y `categoria_id → nombre`. La palabra necesita la referencia, no repetir los metadatos de la fuente o la categoría en cada fila.

Formalmente, una relación está en 3FN si para cada dependencia funcional no trivial $x \rightarrow A$, x es superclave o A es un atributo primo, es decir, parte de alguna clave candidata. La explicación simplificada «cada atributo depende de la clave, de toda la clave y de nada más que la clave» es útil para exposición, pero no sustituye la definición formal en casos especiales.

8.5 Aplicación a entidades del proyecto

Relación	Dependencia principal	Justificación
idioma	<code>id → codigo, nombre</code> ; <code>codigo</code> también es único	Catálogo con identidad propia
fuelle_documento	<code>id → titulo, autor, anio, institucion, url, licencia, nota_uso</code>	La bibliografía depende del documento
acepcion	<code>id → palabra_id, numero_acepcion, definicion, contexto...</code>	Hay UNIQUE sobre palabra y número de sentido
traduccion	<code>id → origen, destino, nota, estado...</code> ; el par es único	El dato describe la relación, no solo una de las palabras
palabra_fuente	<code>(palabra_id, fuele_id) → paginas, nota...</code>	La ubicación documental pertenece al vínculo
progreso_leccion	<code>(usuario_id, leccion_id) → estado, puntaje, fecha</code>	El progreso depende de persona y lección conjuntamente
leccion_regla_gramatical	<code>(leccion_id, regla_id) → orden, nota...</code>	El orden depende del contexto de enseñanza
logro_desbloqueado	<code>(usuario_id, logro_id) → fecha</code>	Evita duplicar la definición del logro

8.6 Anomalías que evita

Actualización: cambiar el nombre de idioma en un solo catálogo evita recorrer miles de palabras. Inserción: se puede registrar una fuente antes de asociarla a una lección. Eliminación: quitar un favorito no elimina la palabra; quitar un vínculo de lección no elimina automáticamente la definición del idioma. La combinación de descomposición y FK conserva referencias válidas.

En el ejemplo de `leccion_palabra`, las tablas se pueden volver a unir por PK/FK para recuperar título y texto sin almacenar esos atributos en la asociación. Esta propiedad hace útil la descomposición; separar campos arbitrariamente sin preservar significado y relaciones no es normalizar correctamente.

8.7 Límites de la afirmación de 3FN

No es riguroso certificar que las 44 tablas están estrictamente en 3FN solo por inspeccionar sus PK y FK.

Hace falta acordar todas las dependencias de negocio. Se observaron decisiones de compatibilidad y datos derivados que deben explicarse:

- `texto_normalizado` y `texto_busqueda` materializan transformaciones del texto para consulta. Introdúcen redundancia controlada y requieren reglas de actualización.
- `usuario.xp`, `rachas` y `última actividad` son agregados operativos. Guardarlos mejora lecturas, pero obliga a mantener coherencia con eventos de progreso; un agregado no viola automáticamente 3FN dentro de una tabla, aunque sí puede duplicar información derivable entre tablas.
- `palabra.fuente_id` coexiste con `palabra_fuente`, y `traduccion.fuente_id` con `traduccion_fuente`. Se pueden interpretar como fuente principal frente a fuentes adicionales, pero esa semántica debe definirse y mantenerse: la existencia de ambas estructuras no la garantiza.
- `palabra.palabra_canonica_id` coexiste con `palabra_canonica`; hay dos representaciones del vínculo que necesitan consistencia.
- `oracion_ejemplo.leccion_id` coexiste con `leccion_oracion`: relación histórica directa frente a reutilización múltiple. Es necesario decidir cuál gobierna cada consulta.
- `actividad_recurso(tipo_recurso, recurso_id)` y `revision_linguistica(tipo_entidad, entidad_id)` son referencias polimórficas. No tienen una FK convencional a todas las tablas posibles. La normalización y la integridad referencial son problemas relacionados, pero distintos.
- No puede asumirse `ciudad` → `pais` sin un catálogo y una identidad geográfica inequívoca: hay ciudades homónimas. Tampoco se debe declarar una violación de 3FN basándose solo en nombres de columnas.

Frase para defensa: «Normalizamos el núcleo por entidades y relaciones, eliminando grupos repetidos y dependencias parciales y transitivas claras. El esquema evolucionado conserva campos derivados y relaciones de compatibilidad; los documentamos en lugar de afirmar una pureza de 3FN que no está demostrada para cada atributo».

9 Persistencia local y operación offline

9.1 Dos bases con responsabilidades distintas

PostgreSQL es el repositorio central compartido. SQLite es el almacenamiento instalado en cada teléfono. Room es la biblioteca que mapea entidades y consultas sobre SQLite; no es un servidor ni otro producto de nube. Esta función de Room está descrita en [Android Room](#).

`AikukisnaDatabase` declara 55 entidades y versión 12. El archivo local es `aikukisna.db`; `DatabaseModule` registra migraciones. El número es mayor que el de tablas remotas porque hay cachés, colas, estado de sincronización, audios y estructuras de evidencia locales. No se trata de dos esquemas idénticos replicados automáticamente por Supabase.

Entre las entidades locales están `palabra_cache`, `traduccion_cache`, `acepcion_cache`, `expresion_cache`, `lecciones`, `cultura`, `reglas`, `memoria de Tuki`, `completar_leccion_pendiente`, selección de traducciones de Cámara y cursores de sincronización. Además, el código activo conserva perfil, favoritos, progreso y otros estados en `SharedPreferences` serializados como JSON. Aunque existan entidades Room relacionadas con usuario, no corresponde afirmar que toda persistencia personal ya migró a Room.

9.2 Cómo arranca sin descargar primero todo el corpus

El APK incluye `supabase_completa.ndjson.gzip`, de 10 923 916 bytes comprimidos en el corte revisado. Es NDJSON comprimido, no una base SQLite preempaquetada que se copie mediante `createFromAsset`.

Al arrancar, `AikukisnaApplication` llama a `SembradorReplicaSupabase`. Este abre el gzip, lee una fila JSON por línea, identifica la tabla remota y la transforma a una tabla local autorizada. Inserta lotes de 500 dentro de transacciones y usa un mutex para serializar el sembrado. En carga inicial ignora filas ya existentes; en actualización remota puede modificar por PK. Un marcador de versión en preferencias se guarda después de comprobar que hay contenido en las tablas esenciales.

El mapa de importación cubre 37 tablas de contenido. No importa indiscriminadamente todas las tablas del servidor ni perfiles de otras personas. Hay además políticas de evidencia Kriol y pobladores de evidencia gramatical y corpus de Cámara.

Este diseño permite disponer del corpus en el primer arranque sin red, a cambio de un APK más grande y un tiempo de preparación local. La disponibilidad de corpus no equivale a iniciar una nueva cuenta sin internet: registro, login remoto y recuperación de contraseña siguen dependiendo de Auth.

9.3 Qué significa offline first en la práctica

La guía oficial de Android recomienda que las lecturas esenciales puedan resolverse localmente y que la red actualice la persistencia sin bloquear ese acceso. [Arquitectura offline first de Android](#).

En Aikukisna hay una combinación concreta de políticas:

Flujo	Sin conexión	Cuando hay red	Observación
Diccionario	Lee Room	Sigue leyendo Room; la sincronización actualiza contenido	Lectura local primero verificable
Traducción textual	Busca oraciones, expresiones y léxico locales	El caso de uso no consulta por sí mismo un traductor remoto	Depende de cobertura del corpus
Cámara de objetos	ML Kit y resolución documental local	El caso de uso intenta primero identificación remota	No es estrictamente local primero en identificación
OCR	Reconocedor local ML Kit	Puede convivir con otras capacidades remotas	Texto reconocido no implica traducción disponible
Tuki	Intenciones, reglas, memoria y contenido local	Usa motor remoto en determinados casos	No hay LLM offline integrado
Dictado español e inglés	Vosk con modelos incluidos	Respaldo remoto si falla la ruta local	Calidad depende del audio y modelo
Dictado Miskito y Kriol	No implementado como reconocimiento local	Transcripción remota mediante proxy	No confundir con traducción textual offline
Pronunciación	Audio local verificado o caché disponible	Puede sintetizar y guardar audio, salvo restricción Miskito	No todo texto tiene audio
Lecciones	Copia local	Varias lecturas intentan red y luego caché	Soporte offline con estrategia híbrida
Perfil y progreso	Datos locales previos	Algunas lecturas intentan servidor primero	No todo cambio tiene cola durable

Flujo	Sin conexión	Cuando hay red	Observación
Finalización de lección	Encola operación y conserva avance	Llama RPC y reintenta pendientes	Requiere endurecer idempotencia y asociación de usuario
Favoritos	Guarda estado y pendientes	Inserta o elimina en servidor	Cola en preferencias
Nuevo login o registro	No autentica nuevas credenciales localmente	Supabase Auth	La sesión previa permite continuar ciertos usos

La afirmación defendible es «la aplicación incluye datos y motores locales para funciones centrales y sincroniza cuando puede». No debe convertirse en «toda función es independiente de internet» ni «siempre se consulta Room antes de cualquier red».

9.4 Cómo se sincroniza

SyncWorker registra una tarea periódica única cada 30 minutos y otra solicitud inmediata, con requisito de red conectada. Android decide el momento efectivo según sus restricciones: 30 minutos no es una garantía de ejecución puntual.

El worker intenta actualizar contenido, enviar lecciones pendientes, enviar favoritos y sincronizar memorias. Si falla, solicita reintento mientras `runAttemptCount < 3`. El flujo de contenido consulta tablas base y sincronizadores especializados, y solo marca la descarga como completa al terminar.

SincronizadorConocimientoLinguistico usa `updated_at` e identificadores como cursor para seis recursos, con páginas de 250. SincronizadorContenidoLecciones sustituye conjuntos locales relacionados dentro de una transacción. SincronizadorReplicaSupabase descarga páginas de 500 de una lista de contenido, 36 tablas públicas y una adicional autenticada. Esa descarga completa no es una suscripción Realtime ni una réplica física PostgreSQL–SQLite.

Los distintos sincronizadores se superponen parcialmente. Esto facilita cubrir estructuras incorporadas en diferentes etapas, pero puede aumentar tráfico y trabajo redundante. La réplica paginada no muestra un protocolo general de tombstones para borrados ni un orden explícito de paginación en ese método. No se puede prometer consistencia exacta ante cambios concurrentes del servidor sin pruebas adicionales.

9.5 Qué ocurre al completar una lección offline

```

Resultado del cuestionario
    ↓
CompletarLeccionUseCase
    ↓
Repositorio intenta RPC si detecta conexión
    ├── éxito → servidor actualiza progreso y XP
    └── sin red o fallo → completar_leccion_pendiente
    ↓
Actualización local de avance perfil y logros
    ↓ vuelve la red
SyncWorker → RPC completar_leccion → borrar pendiente tras éxito

```

La operación remota usa la identidad de `auth.uid()`, hace `upsert` de progreso y actualiza XP y rachas en una función. La fórmula observada es $20 + \max(\text{puntaje}, 0) * 5$; la UI de perfil calcula nivel con bloques de 500 XP. Son decisiones del producto, no estándares educativos.

Hay aspectos que requieren atención: la cola local de lecciones contiene ID, lección, puntaje y fecha, pero no UUID de propietario. La RPC agrega XP en cada invocación y no recibe una clave única de operación. Si el servidor aplica una llamada y se pierde la respuesta, reintentar puede agregar XP otra vez. Por eso, conservar operaciones es útil, pero no demuestra entrega exactamente una vez ni corrección multiusuario en el mismo dispositivo.

10 Implementación de los módulos inteligentes

10.1 Diccionario y Traductor

DiccionarioRepositoryImpl depende de CacheEscriitor y lee datos locales. RepositorioConocimientoImpl consulta coincidencia exacta normalizada, variantes, acepciones y sus relaciones. Para sugerencias calcula Levenshtein sobre candidatas; una sugerencia no se convierte automáticamente en traducción aceptada.

El orden principal de resolverTraduccion es oración exacta registrada, expresión exacta con equivalencia, palabra exacta o variante y traducciones por acepción o léxicas. El caso de uso devuelve una equivalencia única, alternativas si hay ambigüedad o un mensaje de ausencia. Rechaza presentar una composición de segmentos como traducción completa.

La bidireccionalidad se implementa consultando relaciones desde cualquiera de sus extremos y reorientando origen/destino. Al intercambiar idiomas, la UI copia el resultado no ambiguo a la entrada. No se entrena otro modelo al pulsar el botón ni se garantiza contenido para los doce pares dirigidos posibles entre cuatro idiomas.

Existe una política de prioridad por estados: validada, documentada, importada, pendiente y rechazada. Algunas consultas excluyen solo lo rechazado y permiten contenido importado. Por tanto, «está en Room» o «la app dice verificada» no equivale necesariamente a «revisada por una persona hablante». La procedencia, el estado y la revisión humana deben distinguirse.

10.2 Cámara

CameraX obtiene imágenes. ML Kit puede reconocer texto, detectar regiones y producir etiquetas. La capa de dominio toma la identidad detectada y busca una equivalencia documental. Para el corpus controlado, CorpusCamaraRepositoryImpl usa selecciones locales y IDs de palabras; otras etiquetas pasan por búsquedas exactas y traducción.

Cuando hay conexión, ReconocerObjetoUseCase intenta Gemini para identificar el objeto en español, y conserva el respaldo local si esa llamada falla. La traducción de la etiqueta vuelve al diccionario o al corpus Room. En el caso especial de destino español, el código puede devolver directamente la etiqueta española reconocida fuera del corpus controlado: no debe describirse como una validación documental universal de toda etiqueta.

El corpus controlado documentado contempla 30 conceptos. La evidencia previa registró 119 selecciones por idioma: 30 Miskito, 30 español, 29 Kriol y 30 inglés. Esto no prueba reconocimiento universal de objetos; además, reconocer correctamente una imagen y traducir correctamente su etiqueta son dos comprobaciones distintas.

10.3 Tuki

Tuki combina clasificación de intención, respuestas funcionales, contexto real del estudiante, memoria local, recuperación de información y un motor remoto opcional. Las preguntas sobre idioma seleccionado, XP y función del tutor tienen respuestas directas para no tratarlas erróneamente como búsquedas de palabras.

La memoria guarda resúmenes y contexto de conversación; no significa que los pesos de Gemini se reentrenen con cada conversación. El motor local no es un LLM instalado. La generación remota se apoya en contenido recuperado, un patrón de recuperación aumentada sin evidencia de base vectorial o embeddings en este flujo.

El proxy remoto también construye contexto consultando funciones SQL y el perfil. Sus instrucciones buscan evitar invenciones, pero las instrucciones a un modelo no garantizan fidelidad perfecta. Además, su tratamiento de componentes o rutas documentadas y algunas ramas de Tuki no son idénticos a la negativa estricta del Traductor. La defensa debe reconocer la necesidad de pruebas diferenciadas de ambos módulos.

10.4 Voz y pronunciación

Vosk usa modelos incluidos para español e inglés. Los audios se procesan localmente en esos idiomas cuando la ruta tiene éxito; Miskito y Kriol usan transcripción remota si hay conexión. `SpeechRecognitionManager` también integra el reconocedor del sistema para determinados flujos; pedir modo offline a un servicio del sistema no demuestra que todos sus paquetes lingüísticos estén disponibles.

`PronunciationEngineImpl` prioriza audio local verificado y, para Miskito, exige audio humano: si no existe, devuelve no disponible. Para otras lenguas puede usar caché y luego ElevenLabs. Una voz sintética multilingüe no certifica pronunciación nativa Kriol, y leer letras de una palabra no sustituye validación fonética.

10.5 Entrenamiento de traducción

El corpus de entrenamiento documenta, por dirección, 2 752 pares de entrenamiento Miskito–español, 281 Kriol–inglés, 83 Kriol–español y 879 español–inglés, además de validación y prueba separadas. No hay pares directos en las particiones documentadas para Miskito–inglés ni Miskito–Kriol.

El pipeline controla contaminación entre particiones y ambos sentidos, y no fabrica traducciones inversas inexistentes. Los ensayos técnicos pequeños no produjeron un traductor útil; el modelo entrenable es trabajo separado de la app. No se ha incorporado un modelo de traducción neuronal validado en el candidato Android. Completar ese objetivo requiere datos adecuados, entrenamiento, evaluación lingüística y conversión/integración eficiente en dispositivo.

11 Seguridad e integridad

RLS permite condicionar acceso por fila con la identidad de Auth. En los datos personales revisados aparecen predicados como `auth.uid() = usuario_id`; para perfil se compara con `id`. Catálogos y contenidos tienen políticas de lectura pública o autenticada según el recurso. Las revisiones lingüísticas requieren membresía administrativa. La distinción entre permisos de tabla y políticas de fila está documentada en [Supabase RLS](#).

Una política habilitada no prueba seguridad completa. Deben comprobarse roles, privilegios, funciones, campos editables y contexto de cada operación. Las políticas personales observadas de tipo ALL con `WITH CHECK` no explícito usan las reglas de PostgreSQL para el predicado aplicable; no corresponde declarar acceso irrestricto solo porque el catálogo muestre ese campo como nulo.

El trigger `proteger_columnas_gamificacion` conserva XP y rachas ante actualizaciones ordinarias del perfil, salvo que se active la configuración transaccional usada por la RPC. Sin embargo, la RPC de completar lección acepta el puntaje proporcionado y no comprueba allí el umbral pedagógico ni el máximo de 100. Esto debe endurecerse si los puntos necesitan resistencia a clientes manipulados.

Hay claves de proveedor declaradas como campos BuildConfig en Gradle aunque los repositorios activos usan proxies. Un valor secreto colocado allí podría terminar dentro del APK: que `local.properties` esté ignorado por Git no evita su inclusión en el binario. No se inspeccionaron ni reprodujeron valores secretos; la recomendación es mantener claves de proveedor solo en servidor y retirar campos innecesarios en una tarea específica.

El manifiesto permite backup y los XML de copia conservan estructura de ejemplo sin exclusiones específicas. No se observó cifrado de base mediante SQLCipher ni protección integral de todos los SharedPreferences mediante Android Keystore. El aislamiento del sandbox Android es una protección, pero no debe presentarse como cifrado adicional verificado.

Los contenidos empaquetados son extraíbles del APK. Esto es compatible con contenido educativo distribuible, pero exige revisar licencias, consentimiento de audios y qué información puede publicarse. Si se envían imágenes, audio o conversaciones a servicios externos, la experiencia debe informar y aplicar políticas de privacidad apropiadas. Este informe no constituye una revisión legal ni una certificación de seguridad.

12 Pruebas y proceso de construcción

El proyecto dispone de pruebas unitarias Kotlin/JUnit y pruebas Python del pipeline. El candidato del 1 de octubre completó **81 pruebas Android unitarias sin fallos** siguiendo `sync` → `clean` → `tests` → `assembleDebug`, y fue instalado en un Samsung SM-S908U1. Estos resultados se reutilizan de la evidencia existente; no se repitió la compilación para redactar el informe.

`scripts/verify-build.ps1` resuelve dependencias de `debugRuntimeClasspath`, ejecuta `clean`, `testDebugUnitTest` y `assembleDebug`, deteniéndose si falla una etapa. La etapa llamada `sync` es resolución/configuración mediante CLI; no es una afirmación de haber pulsado Gradle Sync en Android Studio. El script respalda `build/test-captures` antes de `clean`.

Hay evidencias previas de respuestas funcionales de Tuki y consultas del Traductor, pero la grabación offline continua del último candidato no quedó aprobada. La última captura se interrumpió para retirar el aviso solicitado. No se debe transformar la instalación correcta o las 81 pruebas en una afirmación de aceptación completa de Cámara y Traductor offline.

El artefacto instalado es debug, versión configurada 1.0 y `versionCode` 1. Existe configuración Gradle para release, minificación y firma condicionada a propiedades, pero no se generó ni publicó una release en esta tarea. El ciclo de validación no equivale a un pipeline de despliegue automático en tienda.

13 Fortalezas y aspectos pendientes

Aspecto	Hecho defendible	Trabajo pendiente o precaución
Arquitectura	Capas, contratos, Hilt y casos de uso	Reducir dependencias concretas de dominio hacia datos
Datos lingüísticos	Identidad, acepciones, evidencia y relaciones multilingües	Unificar semántica de validación y revisar contenidos importados
Offline	Corpus incluido, Room, modelos Vosk y colas	Homogeneizar lecturas local primero
Base relacional	PK, FK, UNIQUE, CHECK y RLS	Formalizar dependencias y redundancias de compatibilidad
Sincronización	Trabajo condicionado por red y reintentos	Idempotencia, usuario propietario, borrados y conflictos
Traductor	Equivalencias completas y rechazo de composición no validada	Traducción libre neuronal y evaluación por dirección

Aspecto	Hecho defendible	Trabajo pendiente o precaución
Cámara	Reconocimiento y resolución documental separados	Mayor cobertura y prueba física controlada sin red
Tuki	Contexto, intenciones y respaldo local	Consistencia entre políticas locales y remotas
Seguridad	Proxies, Auth, RLS y protección de campos	Hardening de RPC, secretos de compilación y backups
Evolución	Migraciones locales y scripts SQL	Exportar esquemas Room y completar espejo local de backend

`exportSchema=false` en Room desactiva su exportación automática de esquema. Las migraciones locales existen, pero no se debe asegurar cobertura de pruebas de todas ellas sin evidencia. Solo una de las tres Edge Functions desplegadas tiene código en la carpeta local `supabase/functions` inspeccionada; es conveniente versionar también las otras para recuperación y revisión. Las cinco migraciones SQL locales no son por sí solas una reproducción demostrada de todo el esquema remoto evolucionado.

También hay diferencias semánticas entre servidor y réplica, como la clave que el sincronizador espera para `expresion_contexto_cultural` frente a su PK real. No se modificaron estas áreas al preparar el informe: se registran como riesgos verificables, no como fallos funcionales reproducidos en esta sesión.

14 Argumentos para defender las decisiones

14.1 Por qué Kotlin y Android nativo

Permiten integrar directamente CameraX, ML Kit, permisos, ciclo de vida, Room, WorkManager y SDK de identidad. La elección favorece acceso a capacidades del dispositivo y coherencia con el ecosistema Android. El costo es que no se obtiene automáticamente una versión iOS; tampoco garantiza por sí sola mayor velocidad en cualquier tarea. El rendimiento depende del modelo, consultas, memoria y trabajo en hilos adecuados.

14.2 Por qué Supabase

Concentra base PostgreSQL, autenticación, API generada y funciones de servidor. Reduce la infraestructura propia necesaria para entregar un prototipo funcional manteniendo un modelo SQL relacional y reglas de acceso en servidor. El costo incluye dependencia operativa del proveedor, cuotas, conectividad para servicios remotos y disciplina de migraciones y seguridad.

14.3 Por qué Room y no solo llamadas al servidor

El estudiante puede necesitar consultar contenido sin señal. El almacenamiento local reduce dependencia de latencia y permite conservar trabajo pendiente. La nube sigue siendo necesaria para identidad, actualización compartida y servicios remotos. Room no sustituye PostgreSQL: cada uno resuelve un entorno distinto.

14.4 Por qué separar palabra acepción expresión y oración

La grafía no determina un único significado. Una acepción expresa un sentido; una expresión puede tener significado no composicional; una oración conserva contexto y equivalencia completa. Mezclar todo en una sola estructura dificulta desambiguar y justificar traducciones. Esa separación mejora el modelado, pero no genera por sí misma un traductor capaz de producir frases inéditas.

14.5 Por qué usar fuentes y estados de revisión

Las lenguas con pocos recursos requieren trazabilidad. La fuente permite explicar de dónde salió una equivalencia y su página; el estado distingue importación de validación. Esto facilita revisión y corrección sin presentar una salida generativa como verdad documental. La calidad depende de aplicar esos estados de forma consistente en todas las consultas.

14.6 Por qué conservar tablas puente

Porque los hechos son muchos a muchos. Una palabra puede estar en muchas lecciones y una lección usa muchas palabras. La tabla puente evita duplicar la palabra y permite atributos propios del vínculo. Algunas extensiones son uno a uno, y otras usan referencias polimórficas: no se debe llamar tabla puente N:M a cualquier tabla con dos identificadores.

15 Preguntas de defensa y respuestas sugeridas

¿Cuál es la arquitectura? Android nativo con Compose y MVVM, organizado en presentación, dominio, datos e inyección. Es una aplicación cliente de un backend Supabase, con persistencia local y separación por paquetes dentro de un único módulo Gradle.

¿La app está escrita en Java? El código propio principal inspeccionado es Kotlin. Usa herramientas y bibliotecas compatibles con el ecosistema Java/Android y se ejecuta sobre Android Runtime.

¿Por qué 44 tablas? Porque separan entidades, relaciones, evidencia documental, currículo y progreso personal. El número es consecuencia del alcance y de la evolución; no es una métrica de calidad por sí mismo.

¿Cómo demuestras 3FN? Explicando dependencias y claves con ejemplos como idioma, fuente y progreso, y reconociendo campos derivados y relaciones de compatibilidad. No basta decir que hay muchas FK.

¿Cómo funciona sin internet? El APK lleva un corpus gzip que se importa en Room. Los módulos locales consultan SQLite y algunas operaciones de usuario se guardan para sincronizar más tarde. Registro y ciertos servicios de voz/IA siguen siendo online.

¿Se descarga la base completa de usuarios? No. El mapa de contenido se limita a tablas educativas; los datos personales se manejan bajo la sesión y sus políticas. Compartir corpus y compartir información privada son problemas distintos.

¿Qué ocurre si pierde señal al enviar progreso? Se conserva una operación pendiente y se reintenta. Todavía se debe garantizar idempotencia en servidor para que una respuesta perdida no duplique XP y asociar explícitamente la cola al usuario.

¿El frontend se conecta directamente a PostgreSQL? No mediante un socket de base y credenciales de administrador. Usa HTTPS con SDK, PostgREST, Auth y Edge Functions; PostgreSQL aplica restricciones y RLS.

¿El Traductor es una IA generativa? El motor Android actual recupera equivalencias documentadas. El entrenamiento neuronal es una línea separada que todavía no está integrada como traductor libre en el APK.

¿Por qué no concatenar las traducciones de palabras? Porque equivalencia léxica no determina orden, concordancia, sentido ni morfología de la frase. El caso de uso evita presentar esa composición como traducción completa validada.

¿Cómo evita Tuki inventar? Prioriza respuestas funcionales y conocimiento recuperado, limita el contexto y mantiene respaldo local. Esto reduce el riesgo, pero no permite prometer ausencia absoluta de alucinaciones del motor remoto.

¿Hay IA local? Sí hay inferencia local de visión y reconocimiento de voz en idiomas con modelos incluidos. El tutor local es un motor de reglas y recuperación, no un LLM instalado; son tipos distintos de procesamiento.

¿Toda la información de Room está validada por hablantes? No necesariamente. Hay estados importados y pendientes; documentación y aprobación humana deben distinguirse. La presencia en una tabla no es certificación lingüística.

¿Está lista para producción solo porque compila? No. Compilación y pruebas unitarias verifican una parte. Faltan validaciones físicas offline, endurecimiento de sincronización y seguridad, evaluación lingüística y proceso formal de distribución.

15.1 Guion breve para una exposición

Primero explique el problema educativo y la conectividad intermitente. Después presente la arquitectura de dos niveles: cliente Android con almacenamiento local y backend Supabase. Muestre una traducción como flujo Compose–ViewModel–caso de uso–Room y una operación de progreso como flujo hacia RPC con cola offline. Luego defienda palabra, acepción, expresión y fuentes como entidades distintas. Use `leccion_palabra` para 2FN e `idioma` y `fuentes_documento` para 3FN. Cierre con la evidencia real y los pendientes, sin convertir el objetivo futuro de traducción libre en una capacidad ya instalada.

16 Glosario para estudiar

Término	Significado en este proyecto
APK	Paquete instalable de Android
ART	Runtime que ejecuta el código Android
Composable	Función que describe una parte de la interfaz Compose
ViewModel	Componente que conserva y coordina estado de presentación
Caso de uso	Operación de negocio, por ejemplo traducir o completar lección
Repositorio	Interfaz y/o implementación de acceso a datos del dominio
DTO	Representación de un mensaje de transferencia, normalmente JSON
Entity	Representación persistida en Room
DAO	Contrato de consultas y escrituras locales
PK	Clave primaria que identifica una fila
FK	Clave foránea que exige una referencia válida
UNIQUE	Restricción de unicidad de uno o varios atributos
CHECK	Condición que debe cumplir una fila
3FN	Tercera forma normal, definida mediante dependencias funcionales
RLS	Política de acceso por fila en PostgreSQL

Término	Significado en este proyecto
JWT	Token firmado usado para representar la sesión remota
RPC	Llamada a una función del servidor mediante API
Edge Function	Función de backend desplegada en el entorno administrado
Idempotencia	Repetir una operación no produce efectos adicionales indebidos
Consistencia eventual	Réplicas pueden diferir temporalmente hasta sincronizar
Tombstone	Marca persistente de eliminación para propagar un borrado
STT	Conversión de voz a texto
TTS	Conversión de texto a voz
OCR	Reconocimiento de texto en imágenes
Corpus paralelo	Pares de contenido equivalente en dos lenguas
RAG	Generación apoyada en información recuperada; no requiere necesariamente vectores

17 Evidencias y referencias

El anexo **Diccionario de datos** contiene todas las columnas, tipos, nulabilidad, PK, FK, UNIQUE, CHECK, políticas e índices recuperados del catálogo. `metadatos_supabase_20261001.json` conserva el resultado estructurado sin filas de usuarios ni secretos. Las consultas fueron de lectura; los conteos de distintas consultas no forman una instantánea transaccional única si otros clientes modificaron la base simultáneamente.

Archivos principales usados, expresados respecto de la raíz del proyecto para facilitar búsqueda dentro del informe:

- `gradle/libs.versions.toml`, `app/build.gradle.kts`, `settings.gradle.kts`: plataforma, dependencias y módulos.
- `app/src/main/java/com/aikukisna/app/AikukisnaApplication.kt`: preparación inicial y programación del trabajo.
- `app/src/main/java/com/aikukisna/app/di/NetworkModule.kt`, `RepositoryModule.kt`, `Databasemodule.kt`: conexión e inyección.
- `app/src/main/java/com/aikukisna/app/data/local/Aikukisnadbatabase.kt`, `entity/Entities.kt`, `dao/Daos.kt`: contrato Room.
- `app/src/main/java/com/aikukisna/app/data/local/SembradorReplicaSupabase.kt`: importación inicial y actualización.
- `app/src/main/java/com/aikukisna/app/data/local/OfflineUserDataCache.kt`: preferencias y pendientes personales.
- `app/src/main/java/com/aikukisna/app/data/sync/SyncWorker.kt`, `SincronizadorConocimientoLinguistico.kt`, `SincronizadorReplicaSupabase.kt`: sincronización.
- `app/src/main/java/com/aikukisna/app/data/repository/RepositorioConocimientoImpl.kt`, `DiccionarioRepositoryImpl.kt`, `LeccionRepositoryImpl.kt`: acceso y políticas de datos.

- `app/src/main/java/com/aikukisna/app/domain/usecase/TraducirTextousecase.kt`, `ReconocerObjetousecase.kt`, `TranscribirAudioUseCase.kt`, `Completarleccionusecase.kt`: operaciones principales.
- `app/src/main/java/com/aikukisna/app/domain/assistant/TukiAssistant.kt`: política de tutoría.
- `app/src/main/java/com/aikukisna/app/presentacion/viewmodel/TraductorViewModel.kt`: estado de traducción e inversión.
- `app/src/main/java/com/aikukisna/app/ui/theme/`: colores, tipografía y tema.
- `supabase/functions/login-usuario/index.ts`: login por nombre de usuario.
- Código desplegado consultado de `gemini-proxy` y `synthesizer-voice`: integración real de proveedores.
- `docs/traductor_offline_entrenamiento.md`, `scripts/requirements-traduccion.txt`: alcance del entrenamiento.
- `docs/evidencia_funcional/`, `docs/entrega_limitada_20261001.md`: evidencia y alcance del candidato.
- `scripts/verify-build.ps1`: secuencia de construcción exigida.

Las fuentes oficiales citadas explican las tecnologías; las afirmaciones particulares sobre Aikukisna se basan en el código y los metadatos de esta fecha. Este informe no sustituye pruebas de aceptación, una revisión de seguridad completa ni validación lingüística por especialistas.

[Consultar el diccionario completo de datos](#)